

Decision Trees

Date: February 26, 2026

For the next several lectures, we'll be talking about a variety of machine learning models that all build off of the same base: *decision trees*. Decision trees can be used for both classification and regression, and are the basis for a number of other models, including random forests and gradient boosted trees.

To illustrate the basic idea behind decision trees, let's consider a simple classification problem. Suppose you want to train a model to predict whether a patient is at risk of heart disease ten years in the future. For simplicity and ability to visualize things, let's assume that there are only two features: the patient's cholesterol level and their blood pressure. To be clear, predicting a label of +1 means we believe the patient is indeed at risk of heart disease in the future based on their current cholesterol and blood pressure.

Rather than drawing a dataset and showing how decision trees work, it's actually easier to just give an example decision tree and understand what it's doing. Here's a possible decision tree for this task:

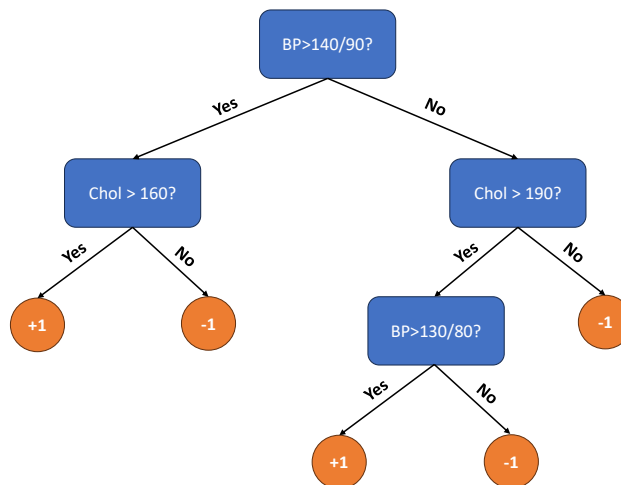


Figure 1: A decision tree predicting heart disease risk based on cholesterol and blood pressure.

To use this decision tree to make predictions, we simply start at the root and answer the question asked at each node. At the root, we check if the patient has blood pressure $> 140/90$. If yes, we go left in the tree, and if no we go right. At the left node, we check if the patient's cholesterol is > 160 . If yes, we predict the patient is at risk, and if no we predict they are not. Going right from the root (if the patient did not have high blood pressure), we check if the patient's cholesterol is *very* high. If the patient's cholesterol is very high, we check their blood pressure against a slightly lower threshold. Otherwise, we predict negative.

In other words, this tree predicts someone is at risk if their blood pressure is very high and cholesterol is moderately high, or if they have very high cholesterol and moderately high blood pressure. This

is a pretty reasonable rule, and it's easy to see how we could have come up with it. It also highlights one of the main advantages of decision trees: they're reasonably interpretable! It's very easy to see how the model is making its predictions, and we can even visualize the decision boundaries it's learning.

In fact, here's one way to draw the decision tree's boundaries in 2D space, where yellow regions correspond to $+1$ predictions and green correspond to -1 predictions:

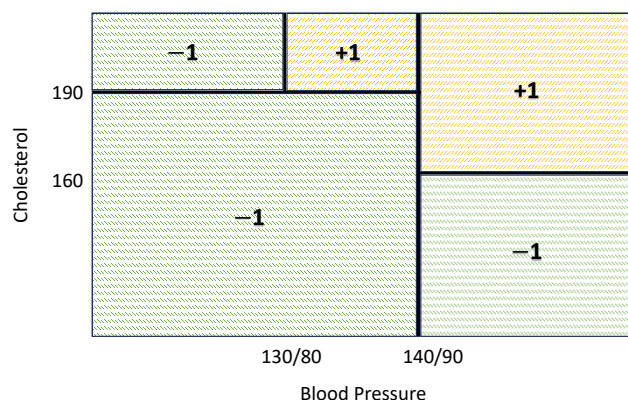


Figure 2: The decision boundaries of the decision tree in Figure 1.

To see how the two figures are equivalent, just consider any point in the plot above and push it through the decision tree in Figure 1. In particular, you may find it helpful to consider one point in each of the obvious “boxes” above.

1 Training Classification Trees

Now that we've seen an example of what a classification tree looks like, how do we train one? Suppose instead of having the decision tree in Figure 1, we just had a labeled training dataset:

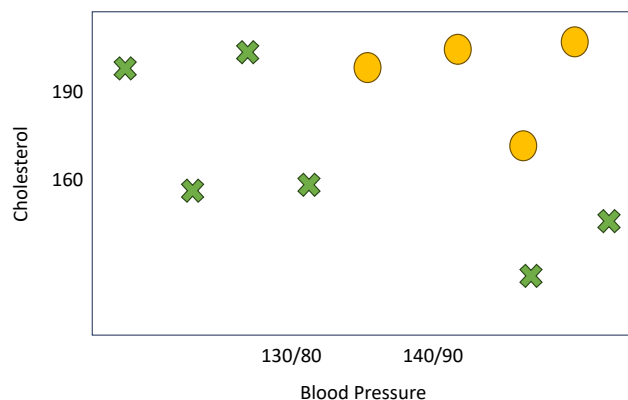


Figure 3: A labeled training dataset.

At a high level, to train a classification tree, we first decide what *split* to make at the root node.

A *split* is defined by a condition (e.g., $\boxed{\text{BP} > 140/90?}$). We call it a split because it “splits” the data into two new datasets: the dataset that satisfies the condition (e.g., the “yes” branch), and the dataset that doesn’t satisfy the condition (e.g., the “no” branch). We then recursively train a classification tree on each of these datasets.

In other words, after the first split we make separates the original dataset into a “yes” dataset and a “no” dataset, we separately train a classification tree on the “yes” branch and one on the “no” branch. This process continues until we reach a *leaf node*, which is a node that does not split the data any further. At this point, we simply predict the majority label of the data in the leaf node. In a “full depth” decision tree, a dataset becomes a leaf node when it is either *pure*—it contains data of only one label—or when it becomes impossible to split the data any further.

The above algorithm is essentially complete with only two things left for us to define:

1. Given a dataset, what splits do we consider at the root node?
2. Given a set of candidate splits, how do we choose between them?

We’ll answer both of these questions in the next two sections. Note that, because the algorithm is recursive, we really only have to consider the *first* split we would make given a dataset. The rest of the splits are handled by the recursive calls. This is a really compelling property: given a dataset, we only have to decide what the first split should be. The rest of the tree is trained for us!

1.1 Choosing Splits

So, given a labeled training dataset, what splits should we consider. The short answer: all of them! Think about the labeled training dataset above. Suppose for a moment that we know we want to split on blood pressure first. How many unique splits are there *actually* for blood pressure? While it’s tempting to say that there are infinitely many lines we could draw, it’s really only between actual training examples that we can draw lines and actually change the “yes” and “no” sub-datasets we end up with. In other words, there are only $n - 1$ possible splits for a dataset of size n .

To illustrate this, consider the following splits we could make on blood pressure for the dataset in Figure 3:

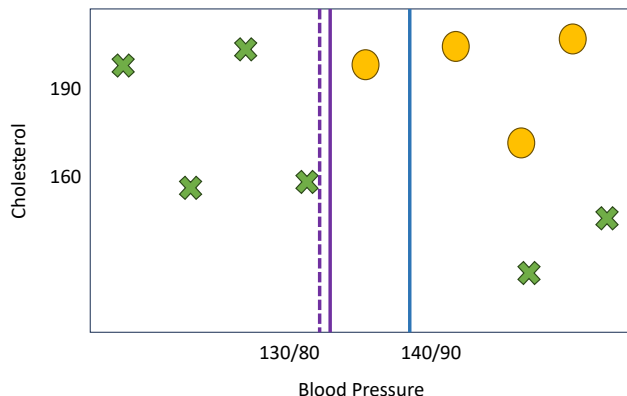


Figure 4: Three possible splits on blood pressure for the dataset in Figure 3.

I'll make two claims about the splits in the figure above:

1. The dashed purple and the solid purple split are almost the same in the sense that the left and right datasets they produce are the same. The only difference is that the solid purple split seems to have higher “margin.”
2. The blue split truly is different from the other two. It produces a different “yes” dataset and a different “no” dataset.

The point about the “margin” in the first item is particularly interesting: if the dashed purple and solid purple splits produce the same “yes” and “no” datasets, intuitively the solid purple split seems “better”. This is because it seems like it will be more robust to noise in the data. In other words, if we were to add a new point to the dataset that was very close to the dashed purple line, it would change the “yes” and “no” datasets. On the other hand, the solid purple line seems like it would be more robust to noise.

Distilling this down, to choose a split, we consider the half way point between subsequent data points along each feature. In other words:

1. For feature j , sort the data points $\mathbf{x}_1, \dots, \mathbf{x}_n$ in increasing value of that feature, so that $\mathbf{x}_{1,j} \leq \mathbf{x}_{2,j} \leq \dots \leq \mathbf{x}_{n,j}$.
2. The set of splits we consider for this feature are $\frac{\mathbf{x}_{i,j} + \mathbf{x}_{i+1,j}}{2}$ for $i = 1, \dots, n - 1$.

How many splits do we consider in total? Well, the dataset has d features, and for each feature we consider $n - 1$ midpoints. Therefore, we consider $O(nd)$ splits in total. That's pretty manageable, all things considered!

1.2 Choosing *Between* Splits

So we're going to try every single split. How do we decide if one split is better than another? The same way we solve every problem in machine learning: we define *loss functions* that tell us quantitatively how “good” a split is. Of course, unlike with previous algorithms, we won't be taking gradients of the loss function or anything like that. Instead, what we'll do is try all possible splits and pick the ones with the lowest loss.

Some notation. To make things clear, first let's recall that we only need to consider a single split. More formally, we have a dataset $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$. A split $\mathbf{s} = (j, v)$ is a choice of feature index $j \in [1 \dots d]$ that we will split on and a value v for that feature that determines the split threshold. The result of the split is two datasets, $\mathcal{D}_{<\mathbf{s}}$ and $\mathcal{D}_{>\mathbf{s}}$, where:

$$\begin{aligned}\mathcal{D}_{<\mathbf{s}} &= \{(\mathbf{x}_i, y_i) \in \mathcal{D} : x_{ij} < v\} \\ \mathcal{D}_{>\mathbf{s}} &= \{(\mathbf{x}_i, y_i) \in \mathcal{D} : x_{ij} > v\}\end{aligned}$$

Without the math: $\mathcal{D}_{<\mathbf{s}}$ is the set of training examples whose j th feature value is less than v , and $\mathcal{D}_{>\mathbf{s}}$ are the ones whose j th feature value is greater than v . In other words, $\mathcal{D}_{<\mathbf{s}}$ is the set of points that the split $\mathbf{s}$ sends left, and $\mathcal{D}_{>\mathbf{s}}$ are the set of points that the split $\mathbf{s}$ sends right. Note that we don't have to consider the case of equality: by definition, the splits are half way between data points, so no data point has a feature value of exactly v for the j th feature.

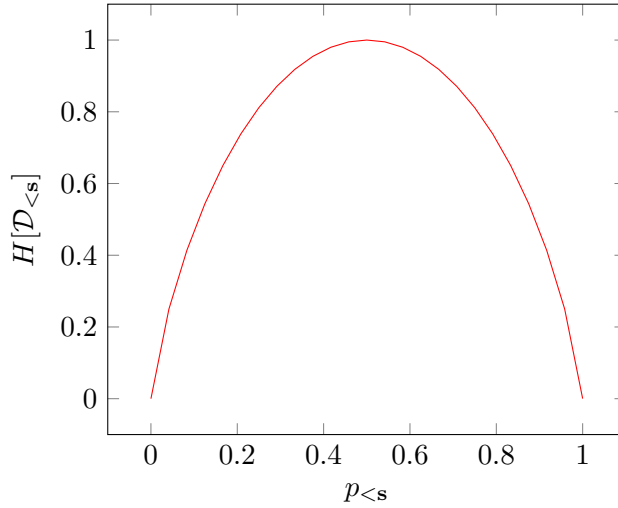
1.2.1 Loss functions.

Suppose we make a split $\mathbf{s}$ for a dataset $\mathcal{D}$, resulting in datasets $\mathcal{D}_{<\mathbf{s}}$ and $\mathcal{D}_{>\mathbf{s}}$. Denote by $p_{<\mathbf{s}}$ the fraction of points in $\mathcal{D}_{<\mathbf{s}}$ with label $+1$. Similarly, $p_{>\mathbf{s}}$ is the fraction of points in $\mathcal{D}_{>\mathbf{s}}$ with label $+1$. Here are some loss functions:

Entropy. The Entropy of the labels in $\mathcal{D}_{<\mathbf{s}}$ is given by:

$$H[\mathcal{D}_{<\mathbf{s}}] = -p_{<\mathbf{s}} \log p_{<\mathbf{s}} - (1 - p_{<\mathbf{s}}) \log(1 - p_{<\mathbf{s}})$$

What is this doing? Here's a plot of this as a function of $p_{<\mathbf{s}}$.



The entropy of the dataset $\mathcal{D}_{<\mathbf{s}}$ takes its minimum value when $p_{<\mathbf{s}} = 0$ **or** $p_{<\mathbf{s}} = 1$. This corresponds exactly to the case when the dataset $\mathcal{D}_{<\mathbf{s}}$ is pure! Either all of the labels are $+1$ ($p_{<\mathbf{s}} = 1$), or they are all -1 ($p_{<\mathbf{s}} = 0$). In fact, the entropy is maximized when there are equal numbers of positive and negatively labeled points ($p_{<\mathbf{s}} = 0.5$).

Extending this idea, the quality of a split as a whole can be measured by the entropy of both datasets that result, $H[\mathcal{D}_{<\mathbf{s}}]$ and $H[\mathcal{D}_{>\mathbf{s}}]$. The only catch is the *number* of data points in each dataset. Consider an extreme example where the split $\mathbf{s}$ results in a single data point in $\mathcal{D}_{<\mathbf{s}}$ but 1000 data points in $\mathcal{D}_{>\mathbf{s}}$. Obviously, $\mathcal{D}_{<\mathbf{s}}$ is pure: it only has one data point. $\mathcal{D}_{<\mathbf{s}}$ could be maximally impure though: 500 $+1$ labels and 500 -1 labels. Clearly, the entropy of $\mathcal{D}_{>\mathbf{s}}$ should dominate the quality calculation. To achieve this, we just take a **weighted average**:

$$H[\mathbf{s}] = \frac{|\mathcal{D}_{<\mathbf{s}}|}{|\mathcal{D}|} H[\mathcal{D}_{<\mathbf{s}}] + \frac{|\mathcal{D}_{>\mathbf{s}}|}{|\mathcal{D}|} H[\mathcal{D}_{>\mathbf{s}}]$$

This is just the entropy of both datasets that result from the split $\mathbf{s}$, weighted by the number of data points that end up on each side. To pick a split, we just pick the one that minimizes entropy:

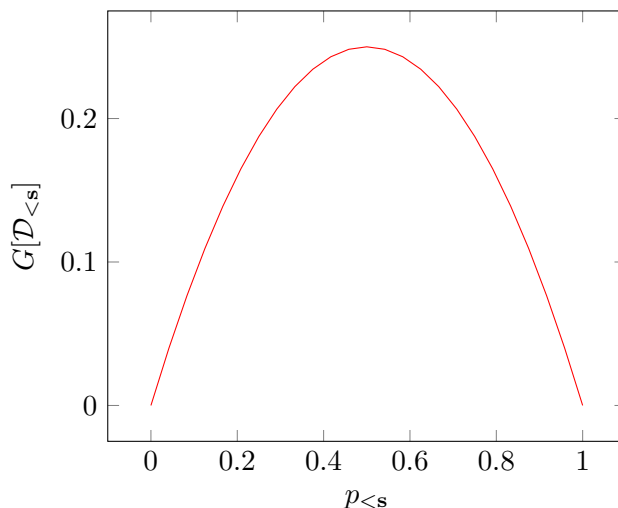
$$\mathbf{s}^* = \arg \min_{\mathbf{s} \in \mathcal{S}} H[\mathbf{s}]$$

Here, $\mathcal{S}$ is the set of all possible splits that we discussed in the previous section.

Gini index. The Gini index of the labels in $\mathcal{D}_{<\mathbf{s}}$ is given by:

$$G[\mathcal{D}_{<\mathbf{s}}] = p_{<\mathbf{s}}(1 - p_{<\mathbf{s}})$$

That looks like this:



which looks extremely similar to Entropy. The Gini index is again maximized at $p_{<\mathbf{s}} = 0.5$ with value $0.5 * (1 - 0.5) = 0.25$. The Gini index is probably the most popular splitting criterion because it's so easy to remember.

Just like with entropy, we can define the Gini index of the split $\mathbf{s}$ as the weighted average of the Gini indexes of the resulting datasets:

$$G[\mathbf{s}] = \frac{|\mathcal{D}_{<\mathbf{s}}|}{|\mathcal{D}|} G[\mathcal{D}_{<\mathbf{s}}] + \frac{|\mathcal{D}_{>\mathbf{s}}|}{|\mathcal{D}|} G[\mathcal{D}_{>\mathbf{s}}]$$

1.3 Base cases

To summarize so far: to build a decision tree, we start from the root with the full training dataset $\mathcal{D}$. We consider all possible $O(nd)$ splits we could make, and pick the one that minimizes some loss like weighted Gini index or weighted Entropy. This results in some split $\mathbf{s}^*$ and datasets $\mathcal{D}_{<\mathbf{s}^*}, \mathcal{D}_{>\mathbf{s}^*}$. We then recurse on both of these datasets, pretending they are their own independent, original datasets.

When do we stop splitting? Generally speaking there are two possibilities (both possibilities also apply to $\mathcal{D}_{>\mathbf{s}^*}$):

1. Eventually during our recursion, $\mathcal{D}_{<\mathbf{s}^*}$ will only have one data point in it. If that happens, it'd be pretty tough to keep splitting! We just assign that node in the tree the label of that data point.
2. At some point, it might become impossible to split further. This can only happen when the remaining data points in $\mathcal{D}_{<\mathbf{s}^*}$ are identical (**think about why**). When this happens, we just take the mode (majority vote) label as the label of that leaf.

2 Training Regression Trees

Now that we've seen how to use trees for classification, adapting to regression is very simple. To train, we apply the same basic concept: we consider all possible splits $\mathbf{s}$ and choose the one that minimizes some loss function $\ell(\mathbf{s})$. The only thing that changes is the loss. One option is the weighted squared loss:

$$\ell_{\text{sq}}[\mathbf{s}] = \frac{|\mathcal{D}_{<\mathbf{s}}|}{|\mathcal{D}|} \ell_{\text{sq}}[\mathcal{D}_{<\mathbf{s}}] + \frac{|\mathcal{D}_{>\mathbf{s}}|}{|\mathcal{D}|} \ell_{\text{sq}}[\mathcal{D}_{>\mathbf{s}}]$$

Here,

$$\ell_{\text{sq}}[\mathcal{D}_{<\mathbf{s}}] = \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_{<\mathbf{s}}} (y_i - \bar{y}_{<\mathbf{s}})^2,$$

where $\bar{y}_{<\mathbf{s}}$ is the **average label** in $\mathcal{D}_{<\mathbf{s}}$. The loss for $\mathcal{D}_{>\mathbf{s}}$ is defined similarly. In words, this is the squared difference between the labels in $\mathcal{D}_{<\mathbf{s}}$ and the *average* label in that dataset. Whenever we finish our recursion, the prediction at a leaf node of the decision tree we'll just take to be the average label across all data points remaining in that leaf node.

3 Regularization

If all of your data points are unique, the above process achieves zero training error. This is because, so long as we can keep making splits, every training example eventually ends up in its own leaf node! As we've discussed before, models that achieve zero training error are suspicious: probably we need to regularize them. To regularize decision trees, here are a couple of options:

1. **Bagging.** This is an idea that will lead to another machine learning algorithm, **random forests**. Random forests are such a good idea that we will have an entire lecture (or most of one) on this idea.
2. **Depth limiting.** One of the simplest things you can do to a decision tree to make it less powerful is to simply stop recursing after a fixed depth. In other words, once we reach a depth of k , we simply stop splitting even if the node at that depth is not pure.

As we talk about our next set of machine learning algorithms, bagging and boosting, other ideas in this space will become important as well.